INTERNATIONAL JOURNAL OF RESEARCH IN COMPUTER APPLICATIONS AND ROBOTICS

An Adaptive Intelligent Decision Support System for Real-Time Flood Forecasting

EL MABROUK Marouane[1], KASSARA Omar[1], GAOU Salma[2], ABAJJA Amal[3]

[1] Abdelmalek Essaâdi University, laSIT Laboratory, Faculty of Sciences, Tetuan, Morocco,
elmabroukmarouane@gmail.com / e.elmabrouk.marouane@ieee.org
okassara@gmail.com
[2] Abdelmalek Essaâdi University, LIROSA Laboratory, Faculty of Sciences, Tetuan, Morocco,
salma_gaou@hotmail.fr
[3] Abdelmalek Essaâdi University, Faculty of Sciences and technologies, Tangier, Morocco,
abajja.amal@gmail.com

Abstract

Floods are one of the most costly types of natural disasters in Morocco and other parts of the world. It was reported that the average annual cost of flood damage in Morocco was about 80,000 ha of land, 3,400 houses and 13 schools of damage just in the region of Gharb-Cherarda-Béni Hssen at 2009. However, the effects of flooding can be mitigated, and thereby reduce the loss of life. Thus, the need for a Decision Support System for flood forecasting and warning is essential.
The Decision Support Systems (DSSs) have recently become popular in making decisions related to complex water resource problems. However, the design and the development of some of these applications do not contain all essential elements of a modern-dayDSS, such as effective databases and file management facilities, user-friendly interfaces, etc …

To respond to this need, we propose a DSS for Real-Time Flood Forecasting and warning based on the proposed model in this paper.

Keywords*:* real-time flood forecasting; decision tree; machine Learning; Anytime algorithm; Decision Support Systems.

1. Introduction

The growth of natural and technological hazards is a worldwide phenomenon that comes as a result of the industrialization and the increasing density of occupation of risk areas, prone to hazards or hazardous events. Risks, therefore, pose a global challenge for the future and they're one of the major problems that sustainable development may face.

In terms of natural hazards, Morocco, by its geographical location, is exposed to climatic, meteorological, geological and biological changes which may lead to major risks such as floods, flash floods, earthquakes and landslides etc.

Here comes the need for a kind of decision support in doing the real-time flood forecasting which may give the opportunity to decide before the normal end of treatment because the current algorithms don't provide the answer until the end of treatment. The former will ensure the quality of the decision accuracy of data, the speed of learning time and the execution and response. Therefore, in this paper, we propose a new algorithm that optimizes the execution time, the precision of the decision and response time. We based our approach on Anytime algorithms to produce partial decisions before the end of treatment trees.

2. State of art of the algorithms of constructing the decision trees

The majority of algorithms dealing with decision trees construction follow the same principles; with exceptions concerning the ability of each to provide binary trees or n-ary, the processing capacity of input attributes in terms of density and quality of accuracy.

Some algorithms are characterized by the rapidity of providing answers on small samples such as CART, some are characterized by quick decisions regarding means samples such as C4.5 and ID3 while others are characterized by their rapidity of large samples as CHAID.

The problem of these algorithms is that their performance decreases when the size of the database processed and the number of classes increase. In the next section, we present the basic decision algorithms make in literature.

2.1 Classification and Regression Trees (CART)

CART (Classification and Regression Trees), is characterized by the fact that it constructs binary trees, namely each internal node has exactly two outgoing edges whereas both ID3, C4.5 algorithms generate the decision trees with variable branches per node. CART is unique from other Hunt's based algorithm as it is also used for regression analysis with the help of regression trees. The regression analysis feature is used in forecasting a dependent variable (result) given a set of predictor variables over a given period of time. The CART decision tree is a binary recursive partitioning procedure capable of processing continuous and nominal attributes both as targets and predictors. In CART trees are grown, using genie index for splitting procedure, to a maximum size without the use of a stopping rule and then pruned back (essentially split by split) to the root via cost-complexity pruning. The CART mechanism is intended to produce not one, but a sequence of nested pruned trees, all of which are candidate optimal trees. The CART mechanism includes automatic (optional) class balancing, automatic missing value handling, and allows for cost-sensitive learning, dynamic feature construction, and probability tree estimation.

2.2 Iterative Dichotomiser 3 (ID3)

ID3 (Iterative Dichotomized) algorithm is based on the Concept of Learning System (CLS) algorithm. CLS algorithm is the basic algorithm for decision tree learning. The tree growth phase of CLS is the matter of choosing attribute to test each node by the trainer. ID3 improves CLS by adding a heuristic for attribute selection. ID3 is based on Hunt's algorithm and is implemented serially. This algorithm recursively partitions the training dataset till the record sets belong to the class label using depth first greedy technique. In growth phase of the tree construction, this algorithm uses information gain, an entropy based measure, to select the best splitting attribute, and the attribute with the highest information gain is selected as the splitting one. ID3 doesn't give accurate result when there is too much noise or details in the training data set, thus an intensive pre-processing of data is carried out before building a decision tree model with ID3. One of the main drawbacks of ID3 is that the measure Gain used tends to favor attributes with a large number of distinct values. It accepts only categorical attributes in building a tree model from which algorithm generates various branches per node.

2.3 C4.5

C4.5 algorithm is an improved version of ID3, it uses Gain Ratio as a splitting criteria, instead of taking gain in ID3 algorithm for splitting criteria in tree growth phase. Hence C4.5 is an evolution of ID3. This algorithm handles both continuous and discrete attributes- In order to handle continuous attributes, C4.5 creates a threshold and then splits the list into those whose attribute value is above the threshold and those that are less than or equal to it. Like ID3 the data is sorted at every node of the tree in order to determine the best splitting attribute. The splitting ceases when the number of instances to be split is below a certain threshold. The main advantages of C4.5 is when building a decision tree, C4.5 can deal with datasets that have patterns with unknown attribute values. C4.5 can also deal with the case of attributes with continuous domains by discretization. This algorithm handles training data with attribute values by allowing attribute values to be marked as missing. Missing attribute values are simply not used in gain and entropy calculations. It has an

enhanced method of tree pruning that reduces misclassification errors due to noise or too much detail in the training data set.

2.3 Scalable Parallelizable Induction of decision Tree algorithm (SPRINT)

SPRINT (Scalable Parallelizable Induction of decision Tree algorithm) was introduced by Shafer et al, 1996. It is a fast, scalable decision tree classifier. It is not based on Hunt's algorithm in constructing the decision tree, rather it partitions the training data set recursively using breadth-first greedy technique until each partition belong to the same leaf node or class (Anyanwu et al, 2009 and Shafer et al, 1996). It is an enhancement of SLIQ as it can be implemented in both serial and parallel pattern for good data placement andload balancing (Shafer et al, 1996). In this paper we will focus on the serial implementation of SPRINT. Like SLIQ it uses one time sort of the data items and it has no restriction on the input data size. Unlike SLIQ it uses two data structures: attribute list and histogram which is not memory resident making SPRINT suitable for large data set, thus it removes all the data memory restrictions on data (Shafer et al, 1996). It handles both continuous and categorical attributes.

3. ANY2FC (ANYtime Flood ForeCasting) Model

The algorithms of construction of the decision trees existing in the literature requires the end of learning (the end of construction of the tree) to give a decision, which according to the experimental studies already conducted, as the number of records and classes increases, the execution time also increases, hence the necessity to stop the construction and give partial decisions which may be the final one, is essential. Anytime algorithms provide us this possibility.

3.1 Anytime Algorithm

Anytime algorithm is an algorithm that can find a valid solution to a problem even if it's interrupted at any time before it ends. The algorithm is expected to find better solutions as long as it keeps running. Most algorithms run to completion: they provide a single answer after performing some fixed amount of computation.

However, in some cases, the user may wish to terminate the algorithm prior to completion. The amount of the computation required may be substantial, for example, and computational resources might need to be reallocated. Most algorithms either run to completion or they provide no useful solution information. Anytime algorithms, however, are able to return a partial answer, according to the quality which depends on the amount of computation they were able to perform. The answer generated by anytime algorithms is an approximation of the correct answer.

3.2 New Proposed Model

According to the synthesis presented on the results & comparative studies section, the execution time of the algorithms increases when the number of records is larger and the number of attributes is smaller.

Projecting that's already seen on our application domain which is flooding, according to the large number of records with a small number of attributes in this scope, you can't use any of the existing algorithms in the literature, so you should have an optimal solution that takes into consideration these two constraints. Here comes the need to suggest a new algorithm which is based on the latter. In the following section, we present the ANY2FC Model for real-time forecasting of floods.

ANY2FC (Anytime Flood Forecasting) is a model based on the Anytime's concept to interrupt the treatment before its normal end in order to provide a decision about the impact of flooding. The quality of the decision depends on the quality of the precised shaft portion built in order to save execution time and response to lead to the notion of real-time forecasting.

This model will be based on a parameter called the cost of importance of a factor. Factors are the parameters leading to the incidence of flooding. We distinguish between two types of factors, the dominating factors on a region and the secondary factors.

3.3 Cost of importance

The cost of importance C is a parameter which can stop the growth of the tree to be able to decide. To calculate this parameter we proceed to the following equation:

$$f(C) = \alpha X + \beta \text{ with } X \in]0, +\infty[, X = 1/\text{Value(Factor)} \text{ and } (\alpha, \beta) \in \{0,1\} \qquad (1)$$

3.4 ANY2FC Model

The model is triggered when the cost of importance of the factors start to decrease. If the dominant factors are null then there will be no flood. The system will not be triggered if there is less dominant factor. Here is the algorithm of the model.

ANY2FC Procedure

```
Procedure ANY2FC ()

  Array Factors_List;

  Integer Factor_Type ;

  If ( Number_Of_Important_ Factors () > 1 ) Then

  If ( It's the dominant factor ) Then Factor_Type ← 1;

  Else Factor_Type ← 0;

    If (Decrease_Cost_Factor(Factor_Type, C) = True ) Then

                    Variables: A set of attributes, sample E, class c;
            Initialize an empty tree;
            If ( All the E examples have the same class c ) Then
                    Label the root by c;
            Else  If ( The A attributes set is empty ) Then
                    Label the root by the majority class in E;
                Else
                    Let 'a' best attribute selected in A;
                    Label the root by 'a';
                    Foreach (Value v of a)
                            Building a branch labeled by 'v';
                            Eav is all examples such as e (a) = v;
                            Add the tree constructed by ID3 (A-{a}, Eav, c);
                    EndForeach
                EndIf
            EndIf
            Return root;
      Else

        No Flood;

      EndIf
```

```
    EndIf
EndProcedure
```

Figure 1: ANY2FC Procedure

This function consists of counting the number of factors that have costs of importances in order to trigger the model.

Number_Of_Important_ Factors Function

```
Function Number_Of_Important_ Factors ()
  Number ← 0;
  C ← Null ;
  Dominant_Factor ← Null ;
  Decrease_Cost ← Null ;
  Foreach Factor
    If ( It's the dominant factor on the region ) Then
      C ← αX + β with α ← 1, β ← 1
      Sleep (20 Seconds) ;
      Dominant_Factor ← 1;
      Decrease_Cost← Decrease_Cost_Factor (Dominant_Factor, C);
    If ( Decrease_Cost = True ) Then
        Factors_List [Number] ← Factor ;
        Number ← Number + 1 ;
    EndIf
   Else
    C ← αX + β with α ← 1, β ← 0
    Sleep (20 Seconds) ;
    Dominant_Factor ← 0;
    Decrease_Cost← Decrease_Cost_Factor (Dominant_Factor, C);
    If ( Decrease_Cost = True ) Then
        Factors_List [Number] ← Factor ;
        Number ← Number + 1 ;
    EndIf
```

```
        EndIf
      EndFor
      Return Number;
    EndFunction
```

Figure 2: Number_Of_Important_ Factors Function

This function is used to test the decreasing costs of importances of factors.

Decrease_Cost_Factor Function

```
  Function Decrease_Cost_Factor (Dominant_Factor, C1)
    C2 ← Null ;
    Result ← False ;
    If (Dominant_Factor = 1 ) Then
      C2 ← αX + β with α ← 1, β ← 1
    Else
      C2 ← αX + β with α ← 1, β ← 0
    EndIf
    If ( C2 < C1 ) Then
      Result ← True ;
    EndIf
    Return Result;
  EndFunction
```

Figure 3: Decrease_Cost_Factors Function

3.4 Choosing the best attribute for the decision tree

Gain (with the entropy function i) is also called the Shannon entropy is the criterion with which one can choose the best attribute minimizing gain. Adapting Shannon entropy to our model, we propose the following formula:

$$\mathbf{Hs(C|A) = - \sum i\ P(Xi).\sum k\ P(Ck|Xi).log(P(Ck|Xi)).\sum C(Fi)} \qquad \mathbf{(2)}$$

3.4 System Architeture

In the system architecture, we have two access mode Offline and Online. The offline mode is used to make a pretreatment of all the factors for having the first selection in order to construct the factors table, do the first filtration and treatment to identify important factors next linking the factors to each other as function of others finally the results are stored. However in the Online mode, we load the functions already stored in the Offline mode, we begin the forecast while recording the data in the database. We have two agents in the backgound, one responsable for filtering in deleted factors and the other for filtering in selected factors to reload or to

unreload in real time the factors that have become important or that have become negligible. Then we do a second factors filtration using C parameter while giving a probability of decision for each factor, finally we execute the ANY2FC Model to give the final decision.

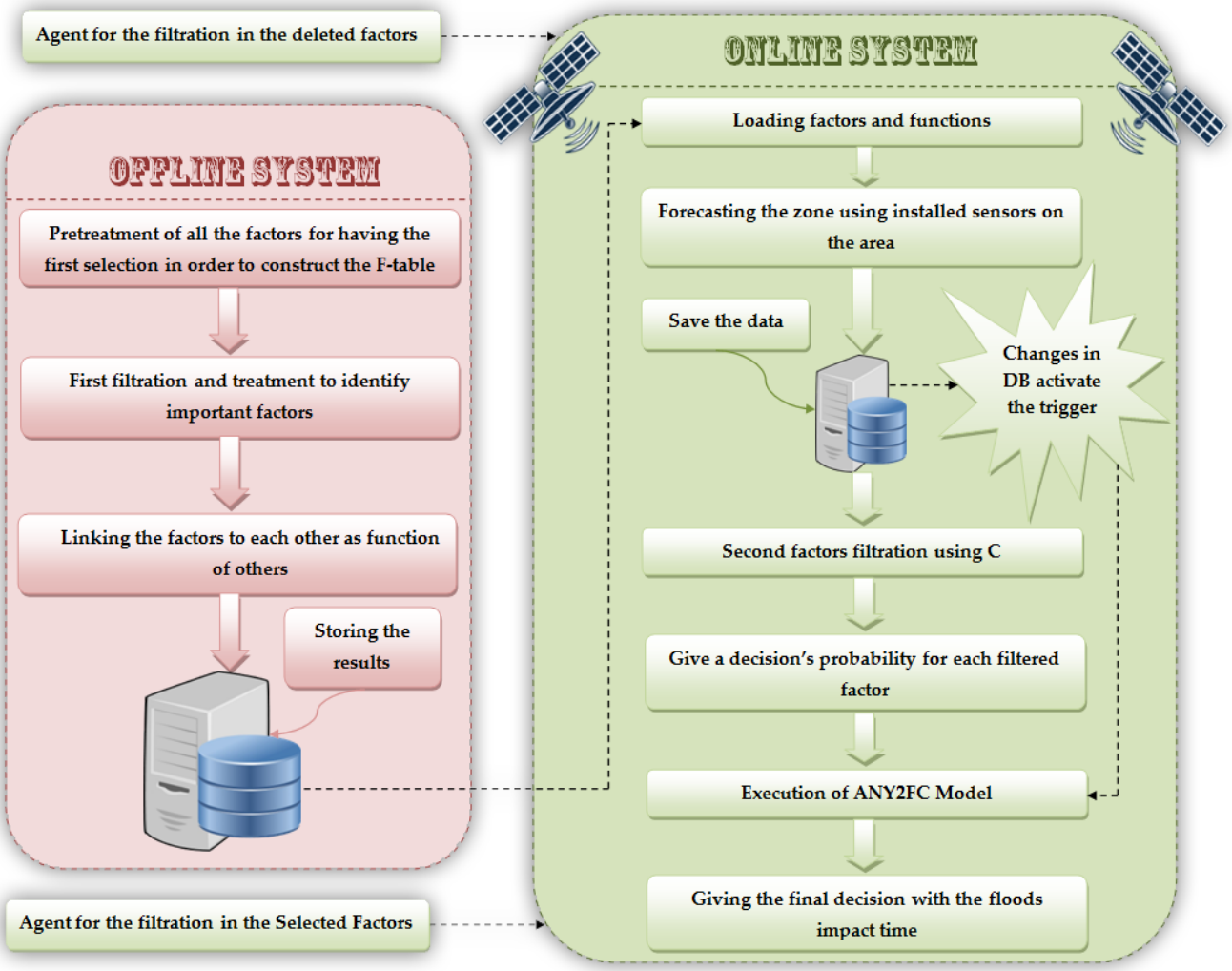


Figure 4: System Architecture

4. Results & Comparative Studies

To compare the algorithm's performance of construction of the decisions trees and our model, we conducted eight tests, each time increased the number of records, the number of classes and the number of attributes in order to identify problems that may appears during execution. Here is the table that shows the experiences already achieved.

Table 1: Training Data

	No. of attributes	No. of records
T_1	6	50
T_2	10	210

T_3	22	456
T_4	36	786
T_5	47	1200
T_6	56	1765
T_7	77	3545
T_8	100	5000

4.1 Results

Table 2: Execution Time (Seconds)

	CART	ID3	ANY2FC
T_1	0.42	0.12	0.05
T_2	1.73	0.51	0.23
T_3	2.55	0.87	0.35
T_4	3.87	1.68	0.72
T_5	7.31	3.24	1.47
T_6	8.95	4.35	2.72
T_7	12.42	7.04	4.45
T_8	21.66	10.11	8.79

Table 2 shows that our model has the fastest execution time among all the other algorithms of constructing the decision trees, independently of number of attributes and records of the data sets volume. This is closely followed by ID3. The table also showed that execution time for ID3 is faster than CART. However, CART is preferred by researches and scientists as it handles both categorical and continuous attributes while ID3 does not.



Figure 5: Execution Time (Seconds)

Figure 5 shows that for all the decision trees algorithms and our model, the execution time increases as the number of records increases. However, our model is faster than the other algorithms of building the decision trees, also when the number of records reached 12,000 the execution time remains stable.

Table 3: Accuracy Classification (%)

	CART	ID3	ANY2FC
T_1	100	100	100
T_2	97	100	100
T_3	94	99	100
T_4	94	98	100
T_5	93	98	99
T_6	93	98	99
T_7	92	93	98
T_8	89	93	95

Table 3 shows that ANY2FC model has the highest classification accuracy among all the other algorithms of constructing the decision trees, this is followed by ID3. The attribute number and record number affect the classification accuracy of our model but only when it isn't reached the threshold; However after that, the accuracy remains stable and could be better.

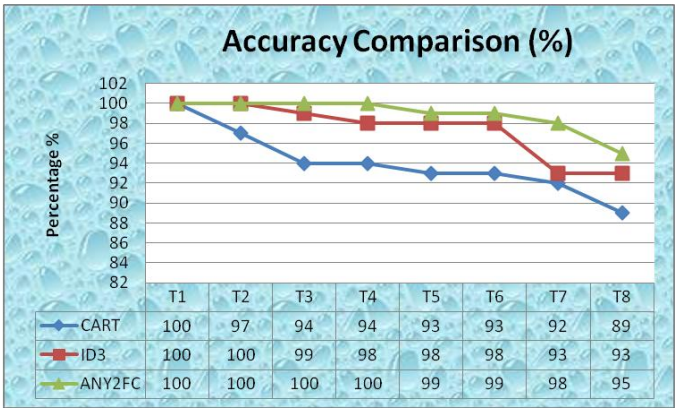


Figure 6: Accuracy Classification (%)

Figure 6 shows that for all the decision trees algorithms and our model, the accuracy decreases as the number of records increases. However, our model is better than the other algorithms of building the decision trees regarding the accuracy, also when the number of records reached 12,000 the execution time remains stable and could be better.

4. Conclusion

In this paper, we have presented the ANY2FC model of real-time flood forecasting that allows reducing the execution time of making the decision when we have big Databases. In addition, we have presented the Cost of importance parameter C and how to calculate it.

In terms of perspectives, it will be studied how to identify the dominant parameters that will be negotiated with the forecasting system to be processed in order to give the final decision. The studies will be based on the principle of computational processing offline and online.

References

A. Srivastava, E. Han, V. Kumar, V. Singh, 1998, "Parallel Formulations of Decision-Tree Classification Algorithms". International Conference on Parallel Processing (ICPP'98). pp 237.

Andrew Colin, 1996, "Building Decision Trees with the ID3 Algorithm", Dr. Dobbs Journal, June 1996.

Breiman, Leo, Friedman, J. H. , Olshen, R. A. , & Stone, C. J, 1984, "Classification and regression trees, Monterey", CA: Wadsworth & Brooks/Cole Advanced Books & Software, ISBN 978-0412048418.

J.C. Shafer, R. Agrawal, M. Mehta, "SPRINT: A Scalable Parallel Classifier for Data Mining",Proc.of the 22th Int'l Conference on Very Large Databases, Mumbai (Bombay), India, Sept. 1996.

J.R. Quinlin. Induction of décision trees. Machine Learning, vol. 1, pages 81 - 106, 1989.

J. R. Quinlan, 1996, "Improved use of continuous attributes in C4.5" , Journal of Artificial Intelligence Research, Vol. 4, pp. 77-90.

J. R. Quinlan, C4.5: Programs for Machine Learning, Morgan Kaufman, 1993.

J.R. Quinlan, Discovering rules by induction from large collections of examples, D. Michie ed., Expert Systems in the Microelectronic age, pp. 168-201, 1979.

Li_Tianrui, 2007, "Construction of Decision Trees based Entropy and Rough Sets under Tolerance Relation", ISKE-2007 Proceedings part of series: Advances in Intelligent Systems Research. ISBN: 978-90-78677-04 8.

M. Mehta, R. Agrawal and J. Rissanen, "SLIQ: A Fast Scalable Classifier for Data Mining",Proc. ofthe Fifth Int'l Conference on Extending Database Technology, Avignon, France, March 1996.

Ricco RAKOTOMALALA. Arbres de Décision. Revue MODULAD, 2005 N 33, pages 163 - 187.

Shlomo Zilberstein, "Using Anytime Algorithmsin Intelligent Systems". FALL, 1996 N 73, pages 73 - 83.

Xin dong Wu,Vipin Kumar ,J.Ross Quinlan, Joydeep Ghosh,Qiang Yang, Hiroshi Motoda, 2007 , "Top 10 algorithms in data mining" , Spinger- Verlag London Limited ,pages 20 -28.